

Career Guide for Non-Traditional Learners

System Methodology & Technical Documentation

Generated: April 22, 2026 | Platform: RP (E:/rp) | Prepared by: Imthadh Ahamed

This document describes the complete methodology, architecture, and step-by-step workflow of the **Career Guide for Non-Traditional Learners** module within the RP platform. It covers the frontend assessment flow, backend AI-driven recommendation engine, skill-gap analysis pipeline, and personalised six-stage career roadmap generation system.

Table of Contents

- 1. Overview & Purpose
- 2. File & Module Inventory
- 3. System Architecture
- 4. User Assessment Flow
- 5. RAG-Based Course Recommendation Engine
- 6. Career Domain Filtering & Scoring
- 7. Skill-Gap Analysis Pipeline
- 8. Six-Stage Roadmap Generation
- 9. API Endpoints
- 10. Data Schemas & Storage
- 11. End-to-End Workflow Summary
- 12. Key Design Decisions & Assumptions

1 Overview & Purpose

The **Career Guide for Non-Traditional Learners** is a sub-system of the RP platform that helps Sri Lankan students who have not followed a conventional university path to discover suitable degree programmes and build a personalised career development roadmap.

The system is grounded in the recognition that many learners arrive with O-Level (Ordinary Level) or A-Level (Advanced Level) qualifications but without clear guidance on which tertiary programme best aligns with their academic background, language preference, financial constraints, and career ambitions. Artificial intelligence techniques — specifically Retrieval-Augmented Generation (RAG) and a multi-agent orchestration architecture — power the recommendation and roadmap engines.

Core Goals

- Surface academically appropriate and financially feasible course options for non-traditional learners.
- Personalise recommendations using a rich multi-dimensional user profile.
- Identify skill gaps between the chosen course and the learner's target career.
- Produce a concrete, six-stage learning roadmap with actions, resources, and success criteria.
- Support multi-language learners (English, Sinhala, Tamil) and flexible study modes (Online / Onsite / Hybrid).

Target Users

- Students who passed O-Levels but did not sit A-Levels.
- Students who sat or completed A-Levels but are unsure of tertiary pathways.
- Mature students seeking career change or upskilling.
- Learners from lower-income households requiring scholarship-sensitive guidance.

2.1 Frontend Files (Next.js / React)

File Path	Role
<code>client/src/app/career-guide/page.tsx</code>	Main landing page; composes all career-guide sub-components.
<code>client/src/components/career-guide/CareerGuideHeader.tsx</code>	Animated gradient header with navigation links.
<code>client/src/components/career-guide/IntroductionSection.tsx</code>	Static introductory copy about non-traditional pathways.
<code>client/src/components/career-guide/NextStepsSection.tsx</code>	CTA section that opens AssessmentModal.
<code>client/src/components/career-guide/SuccessStories.tsx</code>	Displays three real-world success story cards.
<code>client/src/components/career-guide/AssessmentModal.tsx</code>	Modal that routes user to OLForm or ALForm based on qualification tier.
<code>client/src/components/career-guide/OLForm.tsx</code>	14-field intake form for O-Level students.
<code>client/src/components/career-guide/ALForm.tsx</code>	Extended intake form for A-Level students; includes stream & subject grades.
<code>client/src/components/ugc-course-recommender/CareerQuiz.tsx</code>	12-question 1–5 rating quiz for supplementary career interest profiling.
<code>client/src/services/profile.service.ts</code>	CRUD service for user profiles against the backend REST API.
<code>client/src/utils/userStorage.ts</code>	localStorage helper; persists assessment data under key <code>career_guide_user_data</code> .
<code>client/src/types/profile.types.ts</code>	TypeScript interfaces: <code>ALProfile</code> , <code>CreateALProfileRequest</code> , <code>Recommendation</code> .

2.2 Backend Files (Python / FastAPI)

File Path	Role
<code>backend/api/routes/recommend.py</code>	POST <code>/recommend</code> endpoint; calls orchestrator and maps results to schema.

File Path	Role
backend/api/routes/roadmap.py	POST /roadmap/generate & GET /roadmap/health endpoints.
backend/api/schemas/recommendation.py	Pydantic models: UserProfile, RecommendationResult, RecommendationResponse.
backend/api/schemas/roadmap.py	Pydantic models: CareerGoal enum (30+ values), RoadmapStep, RoadmapRequest/Response.
backend/core/rag/retriever.py	RAG pipeline: build_user_profile(), generate_embedding(), rag_search().
backend/core/rag/nomic_embedder.py	Wraps nomic-ai/nomic-embed-text-v1.5 for dense vector generation.
backend/core/agents/orchestrator.py	Recommendation orchestrator: RAG + scoring + filtering → ranked list.
backend/core/agents/career_intent.py	Maps career goals to allowed course domains; computes domain-mismatch penalty.
backend/core/agents/career_path_agent.py	Stores validated skill requirements & journey steps for 20+ career paths.
backend/core/agents/curriculum_agent.py	Parses course document to extract core skills, focus area, duration, languages.
backend/core/agents/skill_gap_agent.py	Compares curriculum skills vs career requirements; identifies priority gaps.
backend/core/agents/roadmap_planning_agent.py	Produces the six-stage personalised learning roadmap.
backend/core/agents/roadmap_orchestrator.py	Coordinates Curriculum → CareerPath → SkillGap → RoadmapPlanning agents.

2.3 Data Files

File Path	Description
backend/data/raw/CourseData.json / .csv	Master course catalogue (~335 KB JSON); embedded into ChromaDB.
backend/data/raw/Tech_Data_Cleaned.csv	Cleaned technology-focused course data for RAG index construction.

File Path	Description
backend/data/raw/Student Course & Career Path Survey...	Survey responses used for validation and demographic insights.
backend/data/embeddings/chroma.sqlite3	ChromaDB persistent vector store for course embeddings.
backend/data/rag_project/chroma_db/	Secondary ChromaDB instance (rag_project collection).

3 System Architecture

The system follows a **three-tier architecture**: a React/Next.js frontend, a FastAPI Python backend, and a vector-database persistence layer backed by ChromaDB.

Tier	Technology	Responsibility
Frontend	Next.js 14, React, TypeScript, TailwindCSS	User assessment intake, result display, roadmap visualisation
Backend API	FastAPI (Python 3.13), Pydantic v2	Business logic, agent orchestration, REST endpoint exposure
AI / RAG Layer	nomic-embed-text-v1.5, ChromaDB	Semantic vector search over course catalogue
Agent Layer	Pure Python classes	Career intent, curriculum parsing, skill-gap analysis, roadmap planning
Data Layer	ChromaDB (SQLite + HNSW), JSON/CSV files	Persistent vector index, raw course catalogue, survey data

Component Interaction Diagram (Logical)

```
Browser (Next.js)
■ ■ /career-guide page
■ ■ CareerGuideHeader
■ ■ IntroductionSection
■ ■ NextStepsSection ■ ■ ■ ■ AssessmentModal
■ ■ ■ ■ OLForm ■ ■ ■ ■ POST /api/profiles
■ ■ ■ ■ ALForm ■ ■ ■ ■ POST /api/profiles
■ ■ SuccessStories

FastAPI Backend
■ ■ POST /recommend
■ ■ ■ ■ orchestrator.recommend_courses()
■ ■ ■ ■ RAG retriever (Nomic + ChromaDB)
■ ■ ■ ■ career_intent.infer_allowed_domains()
■ ■ ■ ■ Scoring & ranking
■ ■ POST /roadmap/generate
■ ■ RoadmapOrchestrator.generate_roadmap()
■ ■ CurriculumAgent.analyze()
■ ■ CareerPathAgent.infer()
■ ■ SkillGapAgent.identify()
■ ■ RoadmapPlanningAgent.plan()
```

4 User Assessment Flow

The assessment flow is the entry point for the Career Guide. It gathers a rich multi-dimensional learner profile through a branched form workflow.

4.1 Entry Point

The user lands on **/career-guide**. The page renders four components in sequence: CareerGuideHeader, IntroductionSection, SuccessStories, and NextStepsSection. Clicking *Get Started* in NextStepsSection opens the AssessmentModal.

4.2 Qualification Tier Selection (AssessmentModal)

The modal presents two mutually exclusive options:

- Option A — 'O/L Passed Without A/L': Routes to OLForm.
- Option B — 'Sat A/L or With A/L Results': Routes to ALForm.

4.3 OLForm — O-Level Student Intake (14 Fields)

Field Group	Fields Collected
Personal Details	Age, Gender, Native Language, Preferred Study Language
Academic Results	O/L grades: Mathematics, English, Science, ICT; IELTS score; Other qualifications
Career & Preferences	Interest area (7 domains), Career goal, Monthly family income, Funding needs
Logistics	Target completion period, Preferred study method (Online/Onsite/Hybrid)

4.4 ALForm — A-Level Student Intake (14+ Fields)

The ALForm includes all OLForm fields plus additional A-Level specific inputs:

- A/L Stream: Bio Science, Physical Science, Commerce, Arts, Engineering Technology, Bio-systems Technology.
- A/L Subjects (3 required): dynamically rendered based on selected stream.
- A/L Subject Grades: grade per subject.

4.5 Form Submission & Data Persistence

On submission both forms:

- Call ProfileService.createProfile() → POST /api/profiles (backend persistence).
- Call userStorage.saveUserData() → localStorage under key 'career_guide_user_data'.
- Navigate to /course-suggestion, passing the profile for recommendation.

4.6 Supplementary Career Quiz (CareerQuiz)

A 12-question interest survey (1–5 star ratings) is available within the UGC Course Recommender sub-module. It measures interest intensity across 12 domains and can augment the assessment profile.

#	Domain
1	Science & Technology
2	Healthcare
3	Design & Visual Creativity
4	Data Analysis & Numbers
5	Business & Entrepreneurship
6	Arts, Culture & Expression
7	Nature, Environment & Agriculture
8	Hands-on / Practical Work
9	Innovation & Research
10	People & Social Work
11	Urban / Corporate Environment
12	Flexible / Non-traditional Work

NOTE: The CareerQuiz ratings are not currently wired into the /recommend API call. The integration point exists but is marked for future implementation.

5 RAG-Based Course Recommendation Engine

The recommendation engine uses **Retrieval-Augmented Generation (RAG)** to semantically match a learner's profile against a vector-indexed course catalogue.

5.1 Vector Index Construction (Offline)

Before serving requests, an offline indexing script (`scripts/build_index.py`) processes `CourseData.json` and `Tech_Data_Cleaned.csv`:

- Each course document is serialised to a rich text representation.
- `nomic-ai/nomic-embed-text-v1.5` generates a 768-dimensional normalised embedding per document.
- Embeddings and metadata are persisted in ChromaDB (HNSW index, SQLite metadata store).

5.2 Profile-to-Query Translation (`build_user_profile`)

At inference time the user's structured profile dictionary is converted to a **rich natural-language query** by `build_user_profile()` in `retriever.py`. The function injects all 15+ profile fields into a templated paragraph that guides the embedding model toward semantically meaningful vectors.

Example profile text snippet:

Provide course recommendations for a student with the following profile:

```
- Age: 22
- Native Language: Sinhala
- Preferred Study Language: English
- O/L Results: Mathematics-A, English-B, Science-B, ICT-A
- A/L Stream: Physical Science
- Career Goal: Software Engineer
- Monthly Family Income: LKR 50,000-100,000
- Study Method Preference: Online
...
```

The goal is to find the most academically suitable, financially suitable, and career-aligned degree courses.

5.3 Semantic Retrieval (`rag_search`)

- `generate_embedding()` normalises the profile text and produces a query vector.
- ChromaDB HNSW search returns the top-k (default 25) closest course documents by cosine distance.
- Each result carries the course name, metadata dict, raw document text, and L2 distance score.

5.4 Scoring & Re-Ranking (`orchestrator`)

The recommendation orchestrator post-processes the raw RAG results through a multi-factor scoring pipeline:

Factor	Source	Effect on Score
Semantic similarity	ChromaDB distance	Base score (converted from L2 distance)

Factor	Source	Effect on Score
Career domain match	career_intent.py	+boost if course domain aligns with career goal
Domain mismatch penalty	career_intent.py	–30 points if course domain conflicts with goal
Language compatibility	User preferred_language vs. course Study Language	Positive or neutral adjustment
Study method match	User study_method vs. course Study Method	Positive or neutral adjustment
Financial filter	User income / funding vs. course Fees	Filters out unaffordable courses pre-ranking

After scoring, the top final_k (default 10) results are selected. The top explain_top_n (default 5) receive a natural-language explanation string.

6 Career Domain Filtering & Scoring (career_intent.py)

The CareerIntentClassifier ensures that recommended courses are **domain-compatible** with the student's stated career goal, preventing semantically close but career-irrelevant results.

Key Data Structure — CAREER_DOMAIN_MAP

A dictionary maps each CareerGoal enum value to a list of allowed course domain keywords. Examples:

Career Goal	Allowed Domains (examples)
Software Engineer	Software Engineering, Computer Science, IT, Computing
Data Scientist	Data Science, Statistics, Machine Learning, AI
Mechanical Engineer	Mechanical Engineering, Manufacturing, Mechatronics
Civil Engineer	Civil Engineering, Structural, Construction, Infrastructure
Business Analyst	Business, Management, Commerce, Administration
Teacher	Education, Teaching, Pedagogy
Doctor	Medicine, Medical, Healthcare, MBBS
Architect	Architecture, Urban Design, Interior Design

Functions

- `infer_allowed_domains(career_goal) → list[str]`: Returns the allowed domains for a goal.
- `matches_career_domain(course_name, career_goal) → bool`: Substring match of course name against allowed domains.
- `get_domain_penalty(course_name, career_goal) → float`: Returns 30.0 if domain mismatches, else 0.0.
- `get_career_alignment_info(course_name, career_goal) → dict`: Returns match flag and penalty metadata.

7 Skill-Gap Analysis Pipeline

Once a student selects a recommended course, the skill-gap pipeline identifies which industry-required skills the course covers and which are missing.

7.1 CurriculumAgent

Accepts a `SelectedCourse` object (name, department, description, curriculum list) and returns a `CurriculumAnalysis`:

- `core_skills`: list[str] — skills explicitly mentioned in the curriculum.
- `focus_area`: str — primary domain (e.g., 'Software Engineering').
- `languages`: list[str] — programming or human languages taught.
- `duration_years`: float — programme length.

7.2 CareerPathAgent

Looks up the `CareerGoal` enum in a hard-coded industry-validated dictionary of 20+ career paths. Returns a `CareerRequirements` object:

- `required_skills`: list[str] — must-have competencies.
- `optional_skills`: list[str] — nice-to-have competencies.
- `role_description`: str — concise job description.
- `typical_journey`: list[str] — 6–8 sequential career milestones.

7.3 SkillGapAgent

Compares `CurriculumAnalysis.core_skills` against `CareerRequirements.required_skills` using a fuzzy alias-based matching strategy:

- `_match_skill(skill, curriculum_skills) → 'full' | 'partial' | 'none'`.
- Alias dictionary maps variant names (e.g., 'OOP' ↔ 'Object-Oriented Programming').
- `identify()` returns a `GapAnalysis`: `covered_skills`, `missing_skills`, `partial_skills`, `priority_gaps`.
- `_prioritize_gaps()` orders missing skills by their position in `required_skills` (earlier = higher priority).
- `get_gap_severity() → 'low' | 'medium' | 'high'` based on proportion of missing required skills.

7.4 Gap Severity Thresholds

Severity	Condition	Action Recommended
Low	< 30% of required skills missing	Course largely sufficient; minor supplementation needed.
Medium	30–60% missing	Supplement with online courses or bootcamps.
High	> 60% missing	Warning generated; learner advised to reconsider or seek alternative paths.

8 Six-Stage Roadmap Generation

The RoadmapPlanningAgent produces a personalised **six-stage learning roadmap** by combining curriculum coverage, skill gaps, and career journey milestones.

8.1 Stage Definitions

Stage	Name	Focus Key	Purpose
1	Foundation Stage	fundamental_skills	Core prerequisites & foundational knowledge.
2	Skill Development	core_technical_skills	Primary technical skill acquisition.
3	Real-world Readiness	practical_application	Projects, internships, applied learning.
4	Performance Strategies	optimization_and_best_practices	Code quality, patterns, efficiency.
5	Knowledge Expansion	advanced_topics	Specialisation & emerging technologies.
6	Industry Polishing	job_readiness	Portfolio, interviews, certifications.

8.2 Each Roadmap Step Contains

- stage_number: int (1–6)
- title: str — stage name.
- goal: str — what the learner should achieve.
- duration: str — recommended time investment (e.g., '3–4 months').
- description: str — narrative description.
- action_plan: list[str] — concrete tasks.
- resources: list[str] — tools, platforms, or references.
- success_criteria: list[str] — measurable indicators of stage completion.
- icon: str — frontend icon identifier.
- skills_covered: list[str] — skills addressed in this stage.

8.3 Personalisation Logic

The RoadmapPlanningAgent populates each stage by:

- Mapping gap_analysis.missing_skills to the relevant stage based on skill category.
- Incorporating career_requirements.typical_journey milestones into stage action plans.
- Adjusting stage duration based on curriculum.duration_years and existing coverage.
- Adding warnings to the response when gap count exceeds 5 or priority gaps are critical.

8.4 RoadmapOrchestrator — Execution Flow

Step	Agent Called	Output
1	CurriculumAgent.analyze(selected_course)	CurriculumAnalysis (skills, focus, duration)
2	CareerPathAgent.infer(career_goal, user_profile)	Requirements (required & optional skills, journey)
3	SkillGapAgent.identify(curriculum, requirements)	GapAnalysis (covered, missing, partial, priority gaps)
4	RoadmapPlanningAgent.plan(all above)	list[RoadmapStep] — 6 personalised stages

8.5 Response Metadata

- curriculum_focus — primary domain of the selected course.
- languages — languages taught.
- duration_years — programme length.
- skill_gaps_count — total missing required skills.
- gap_severity — 'low' / 'medium' / 'high'.
- priority_skills — top 3 skills the learner must acquire first.

9 API Endpoints

9.1 POST /recommend

Method	POST
Path	/recommend
File	backend/api/routes/recommend.py
Input	UserProfile (Pydantic model)
Output	RecommendationResponse: status, list[RecommendationResult], warnings, errors
Process	1. Converts profile to dict 2. Calls orchestrator.recommend_courses(initial_k=25, final_k=10, explain_top_n=5) 3. Maps raw results to RecommendationResult schema 4. Extracts career_opportunities, study_language, study_method, fees from document text

9.2 POST /roadmap/generate

Method	POST
Path	/roadmap/generate
File	backend/api/routes/roadmap.py
Input	RoadmapRequest: user_profile, selected_course, career_goal
Output	RoadmapResponse: status, list[RoadmapStep], metadata dict, warnings, errors
Process	Delegates entirely to RoadmapOrchestrator.generate_roadmap()

9.3 GET /roadmap/health

Method	GET
Path	/roadmap/health
Purpose	Liveness check — returns status of all career guidance agents

9.4 POST /api/profiles (Frontend Service)

Service	client/src/services/profile.service.ts
Endpoint	POST /api/profiles
Payload	CreateALProfileRequest

Purpose	Persists complete assessment intake data in the backend database
----------------	--

10.1 UserProfile (Pydantic — recommendation.py)

- age: int
- gender: str
- native_language: str
- preferred_language: str
- ol_results: dict — subject → grade mapping
- al_stream: str | None
- al_results: dict | None
- other_qualifications: str | None
- ielts: float | None
- interest_area: str
- career_goal: str
- income: str — monthly family income range
- funding_method: str — Self-funded / Scholarship
- availability: str — Weekdays / Weekends / Both
- completion_period: str — e.g., '2 years'
- study_method: str — Online / Onsite / Hybrid
- current_location: str
- preferred_locations: list[str]

10.2 CareerGoal Enum (roadmap.py — 30+ values)

The enum defines all supported career targets. A representative subset:

- SOFTWARE_ENGINEER • DATA_ENGINEER • FULL_STACK_DEVELOPER • DEVOPS_ENGINEER
- ML_ENGINEER • DATA_SCIENTIST • BUSINESS_ANALYST • PROJECT_MANAGER
- CIVIL_ENGINEER • MECHANICAL_ENGINEER • ELECTRICAL_ENGINEER • CHEMICAL_ENGINEER
- INDUSTRIAL_ENGINEER • NURSE • PHARMACIST • TEACHER
- GRAPHIC_DESIGNER • UI_UX_DESIGNER • MARKETING_MANAGER • ENVIRONMENTAL_SCIENTIST
- NETWORK_ENGINEER • COMPUTER_ENGINEER • ARCHITECT • AGRICULTURAL_ENGINEER
- OTHER

10.3 ALProfile (TypeScript — profile.types.ts)

- Mirrors UserProfile but stored client-side.
- qualificationType: 'OL' | 'AL'

- Includes nested recommendation: Recommendation[] for persisting prior results.

10.4 Local Storage

Storage key	career_guide_user_data
Set by	userStorage.saveUserData() on form submission
Read by	Course suggestion page on mount to pre-populate recommendation request
Cleared by	userStorage.clearUserData() (manual or on logout)

11 End-to-End Workflow Summary

Step 1	Landing	User visits /career-guide. Reads introduction and success stories.
Step 2	Qualification Branching	Clicks 'Get Started' → AssessmentModal appears → selects OL or AL tier.
Step 3	Profile Intake	Fills 14+ field form (OLForm or ALForm). Submits.
Step 4	Profile Persistence	ProfileService.createProfile() POSTs to /api/profiles. userStorage.saveUserData() writes to localStorage.
Step 5	Course Recommendation	Browser navigates to /course-suggestion. UserProfile is POSTed to /recommend. RAG retrieval fetches top-25 courses. Scoring pipeline applies career domain boost/penalty. Top 10 ranked courses returned with explanations.
Step 6	Course Selection	User reviews ranked recommendations (name, match score, career opportunities, fees, language, method). Selects one course.
Step 7	Roadmap Request	Frontend POSTs selected_course + user_profile + career_goal to /roadmap/generate.
Step 8	Roadmap Orchestration	RoadmapOrchestrator sequences four agents: CurriculumAgent → CareerPathAgent → SkillGapAgent → RoadmapPlanningAgent.
Step 9	Roadmap Delivery	Six-stage roadmap returned. Each stage has goal, duration, action plan, resources, success criteria.
Step 10	Roadmap Display	Frontend renders interactive six-stage roadmap. Warnings shown if skill gaps are severe. Priority skills highlighted.

12.1 Stated Assumptions

[A1] Qualification scope

The system currently supports two qualification tiers: O-Level only and A-Level (with or without results). Vocational qualifications (NVQ, TVEC) and foreign qualifications are captured under 'Other Qualifications' but are not separately scored.

[A2] Career goal coverage

The CareerPathAgent contains hard-coded skill maps for 20+ career goals. Goals outside this set fall back to the generic 'OTHER' profile, which uses broadly applicable transferable skills.

[A3] CareerQuiz integration

The 12-question interest quiz in CareerQuiz.tsx is rendered in the UGC Course Recommender sub-module. Its output is not currently passed to the /recommend endpoint; it is available as a UI feature for self-reflection purposes only.

[A4] Embedding model

nomic-ai/nomic-embed-text-v1.5 is run locally (CPU or GPU). The quality of semantic matching is bounded by this model's capabilities. The course index must be rebuilt (scripts/build_index.py) after any catalogue update.

[A5] Financial filtering

Income ranges and course fees are compared as string categories, not numeric values. Precise fee comparison would require structured fee data in the catalogue.

[A6] Language of instruction

The preferred_language field is matched against the course's Study Language field using substring search. Mixed-medium courses may not always match cleanly.

[A7] Roadmap personalisation depth

The RoadmapPlanningAgent uses the gap analysis and career journey data to customise stage content. It does not call an external LLM by default (llm_client=None); LLM-enhanced generation is an optional extension path.

[A8] Geographic scope

Course locations and student preferred locations are Sri Lanka-specific. The system assumes local (LKR) currency for fee comparisons.

12.2 Key Design Decisions

D1. RAG over keyword search: Semantic embedding allows the engine to surface courses with relevant content even when exact keywords differ (e.g., 'BSc in Computing' for a 'Software Engineer' goal).

D2. Domain penalty (−30 pts) rather than hard exclusion: Allows edge cases (e.g., a Business & IT double-major) to still surface while pushing clearly misaligned results to the bottom.

D3. Hard-coded career skill maps: Provides consistent, auditable baseline requirements without LLM variability. The maps are based on industry standards and can be updated without retraining.

D4. Six-stage roadmap structure: Mirrors common apprenticeship/bootcamp pedagogy (Foundation → Skill Build → Practice → Optimise → Expand → Polish), giving learners a predictable progression framework.

D5. Dual persistence (backend DB + localStorage): Supports offline-first continuation and reduces latency on the /course-suggestion page by avoiding a blocking profile fetch.

D6. Qualification branching at intake: Separates O-Level and A-Level forms to avoid presenting irrelevant fields (A/L stream, subject grades) to O-Level students, reducing cognitive load.

End of Document · Generated April 22, 2026 · RP Platform · Career Guide for Non-Traditional Learners